Regularization and dimensionality reduction

Lecture 10

Course of:

Statistics and data analysis

Federico De Guio - Martino Borsato

The curse of dimensionality

- In a **high-dimensional space** (hundred of features), the "volume" increases exponentially
  - Finding statistical significance **requires increasing the training data** exponentially
- The **data becomes sparse**: even with millions of events, the space is mostly empty
  - Distances between points become indistinguishable → algos using distance as a metric struggle
- **Models tend to overfit**, reducing generalization
  - True in particular for complex models
- **Computational cost increases**

- TODAY: ways to mitigate this problem → **avoid the model to be lost in the noise**
  - Needs to focus on the real characteristics of the physics phenomenon
  - Avoid overfitting

A way to handle data complexity: **regularization methods**

- James, cap.6

Regularization: an overview

- The idea of regularization revolves around **modifying the loss function L**
- In particular, add a **regularization term** that penalizes some specified properties of the model parameters

$$L_{reg}(\beta) = L(\beta) + \lambda R(\beta)$$

- where λ is a scalar that gives the weight (or importance) of the regularization term

- Fitting the model using the **modified loss function L_{reg}** would result in model parameters with desirable properties (specified by R)

Ridge regression [L2] $\int |f|^2 < \infty \Rightarrow \sum |\beta|^2 < \infty$

- **Goal: discourage extreme values** in our model parameter → want to **penalize parameter magnitude**
- For simplicity use RSS (residual sum of squares) in a linear model as our loss function and add a regularization term so that:

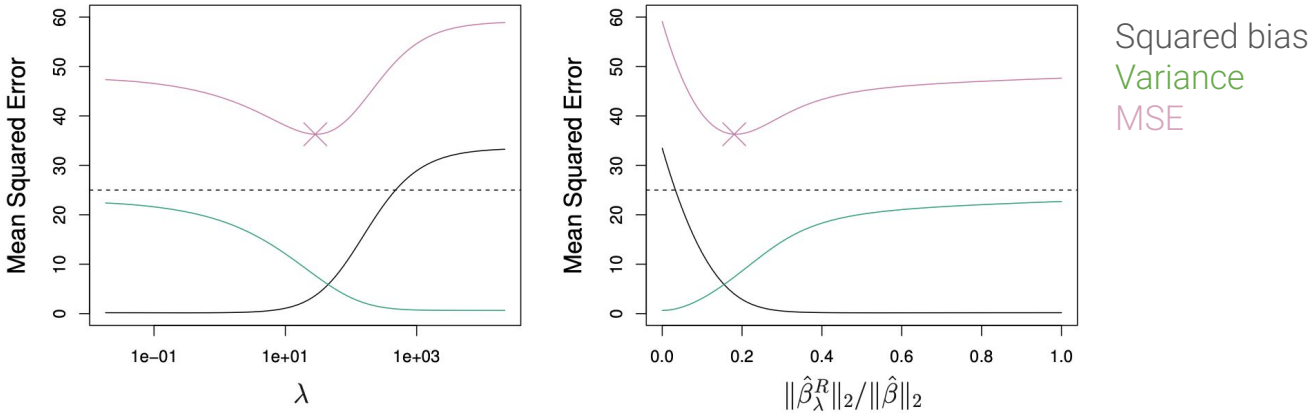
target events features

$$\textbf{LOSS} = \underbrace{\sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2}_{\text{RSS}} + \underbrace{\lambda\sum_{j=1}^{p}\beta_j^2}_{\text{Penalty on the coefficients}} = \mathrm{RSS} + \lambda\sum_{j=1}^{p}\beta_j^2,$$

residual sum square

Advantage in the bias-variance trade-off

- **As λ increases, the flexibility of the ridge regression fit decreases**
  - Decreased variance V↘
  - Increased bias B↗
- However the ridge regression, while reducing the magnitude of the coefficients, **does not exclude any feature**



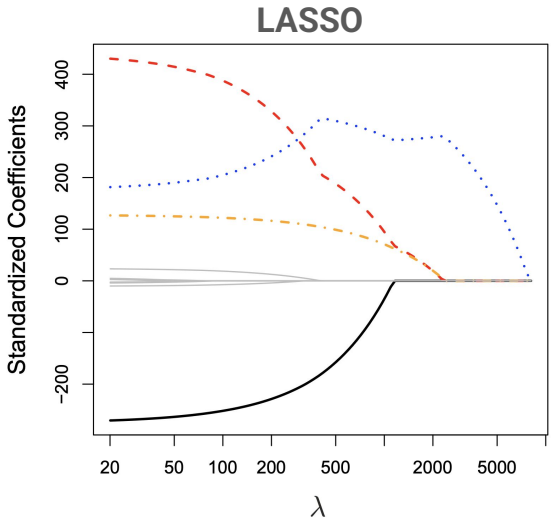
LASSO (least absolute shrinkage and selection operator) regression [L1]

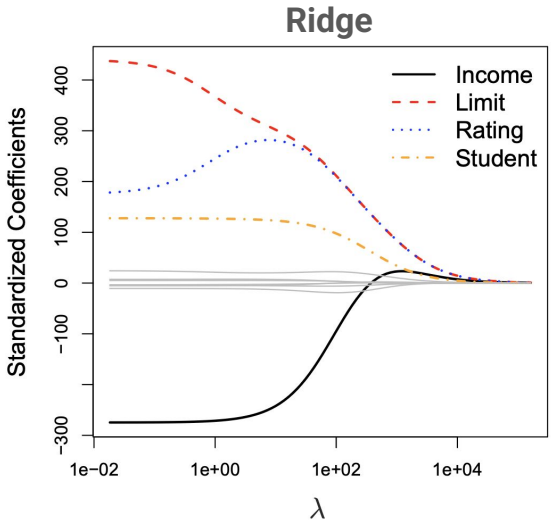
- **Goal**: want to penalize parameter magnitude and completely **exclude irrelevant features**
  - Similar formulation with respect to ridge
  - Involves the absolute value of the coefficients in the penalty term

$$\textbf{LOSS} = \sum_{i=1}^{n}\left(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij}\right)^2 + \lambda\sum_{j=1}^{p}|\beta_j| = \mathrm{RSS} + \lambda\sum_{j=1}^{p}|\beta_j|.$$

Feature selection often means easier interpretation

- Coefficients **go to zero as λ grows** → some of the features become irrelevant
  - Compared to Ridge, LASSO acts as a **feature selector**
- In general LASSO is to be preferred when a small number of features is sufficient. Ridge performs better when the response is a function of many features with ~same importance





Another formulation for Ridge and LASSO

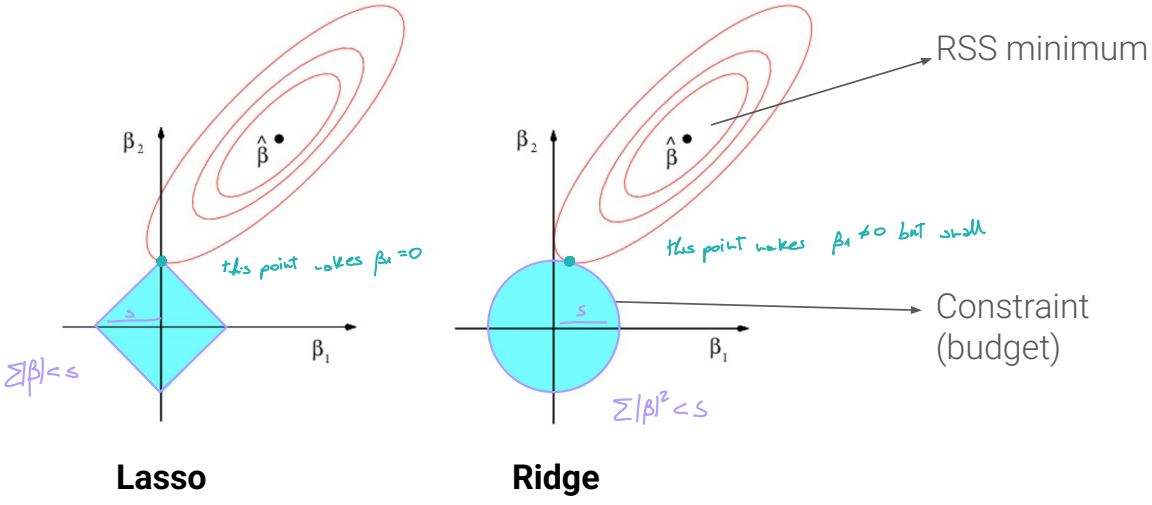
Available budget

Lasso: $$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^{p} |\beta_j| \leq s$$

Ridge: $$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 \leq s$$

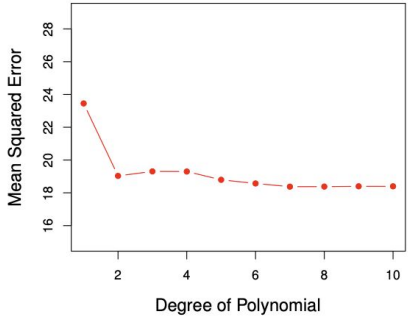
How do Ridge and Lasso select the features?

- Consider a problem with 2 features $\boldsymbol{\beta}_1$ and $\boldsymbol{\beta}_2$
  - Red ellipses are equal RSS contours



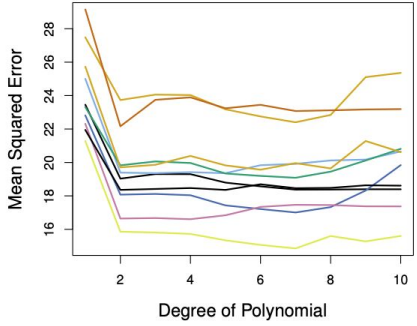
But how is λ chosen? Cross validation

- The tuning parameter λ needs to be optimized → **cross validation** offers a simple way to do so
- In general when performing a training we want to know
  - What is the **best hyper-parameter configuration**
  - How to estimate **the test error for our model**



- Simple strategy:
  - **Split the dataset into two parts**
  - Train on the first, compute the error on the second

- **Careful!** Estimates (error) can vary considerably with different train-validation splits

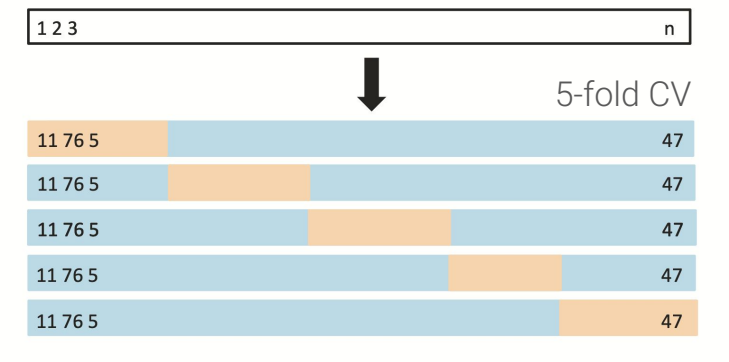


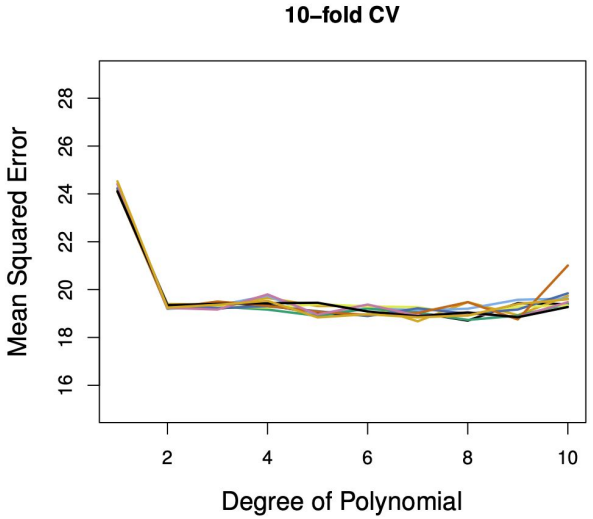
One example: K-fold cross validation

- Split the data into **k subset of folds**
- For every i = 1,..., k:
  - **Train the model** on every fold except the ith fold
  - **Compute the test error** (MSE) on the ith fold
- **Average** the test errors

cross validation

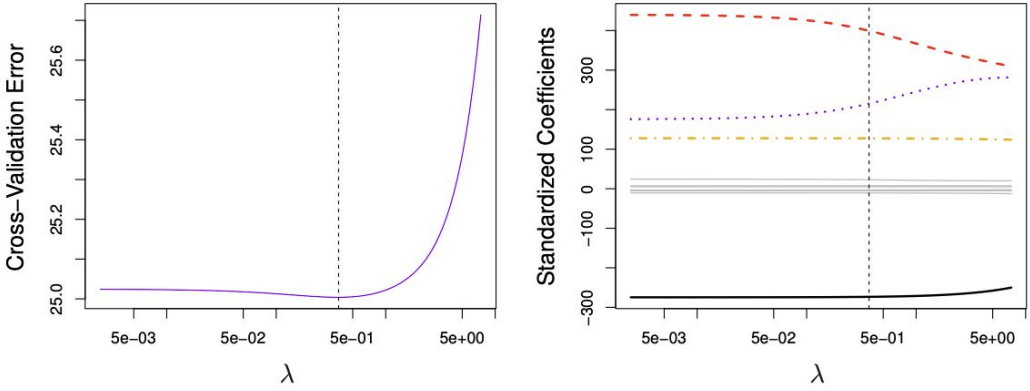
$$\mathrm{CV}_{(k)} = \frac{1}{k}\sum_{i=1}^{k}\mathrm{MSE}_i.$$





Selecting λ by cross-validation

- If our goal is to find the best λ we are interested in **finding the minimum point** in the estimated test MSE curve



Another way to handle the data complexity:
dimensionality reduction

- James, cap.12

The garbage in, garbage out principle

- The quality of a model's **output is directly dependent on the quality of its input** data

- Types of garbage in data:
  - **Inaccurate** data: Erroneous values resulting from poor collection or recording
  - **Incomplete** data: Missing information
  - **Biased** data: Data that is not representative of the target population
  - **Noise**: tends to confuse the model
  - **Outliers**: anomalous data points

- **Data preparation** helps to mitigate problems deriving from not optimal datasets
  - Data cleaning
  - Feature engineering
  - Data validation
  - **Often this is the part of the analysis that takes most of the time**

Good practice for **data preparation**: the standard scaler

- If the input features have different scales, **the model tends to ignore signals from the features spanning a smaller range**

- The standard scaler **transform the data** such that all the features have **mean~0 and sigma~1**

$$z = \frac{x - \mu}{\sigma}$$

- Avoid that a feature dominates only because it has **different ~~units~~** ranges
  - Helps the convergence

- An alternative often preferred when dealing with NN is the **min-max scaler**
  - Compresses the data in a given interval, usually between 0 and 1

What is Principal Component Analysis (PCA)?

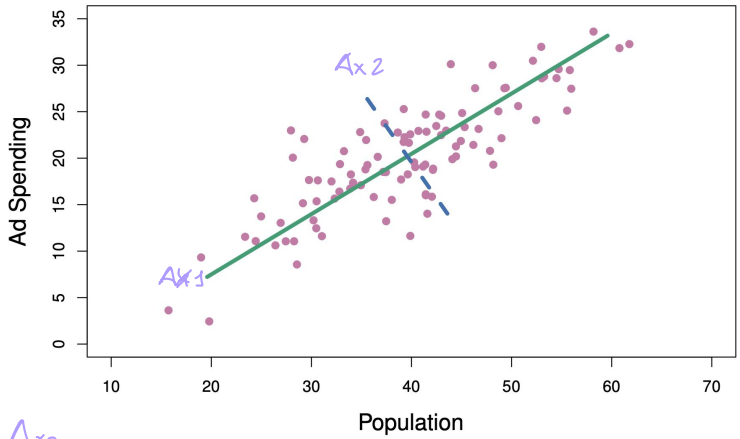
from multiple dimensions it returns a linear plot from a combination of the features

- Probably **one of the first unsupervised procedure** ever (non-parametric method)

- It provides a way to **compress** and visualize high dimensional data, summarizing the most "important" information

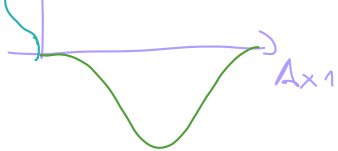
- **The first PC** is the direction of the line that is closest to our data points on average
  - Closeness measure by squared distance

- **Intuition:** Closest line offers the best 1-dimensional compression of our data



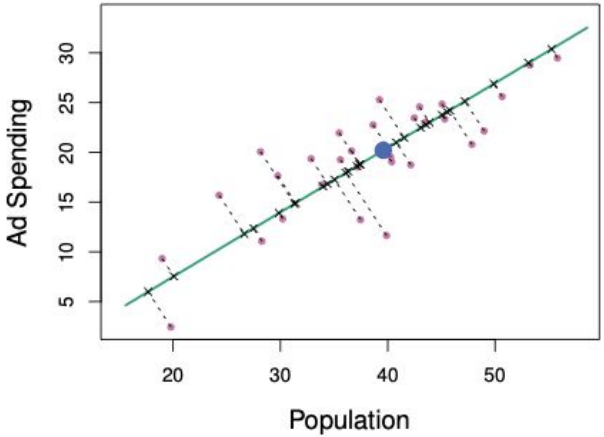
Ax2

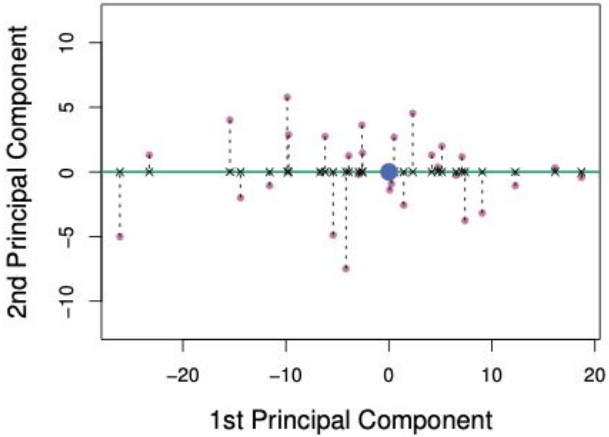
A second interpretation



- **Intuition:** High variance directions are often interesting directions
  - Naively assume **higher SNR** along these directions

Sig to Noise ratio S/N

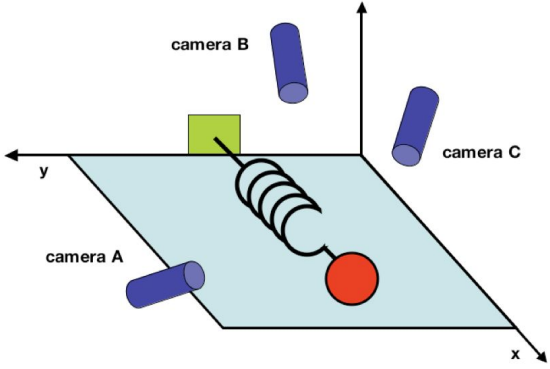




Practical toy example

- Is there **another basis**, a **linear combination** of the original basis, that best re-expresses our data set?

- Let's consider a pendulum
  - Mass attached to a spring moving on a **straight line**
  - 3 cameras observing the scene from different angles

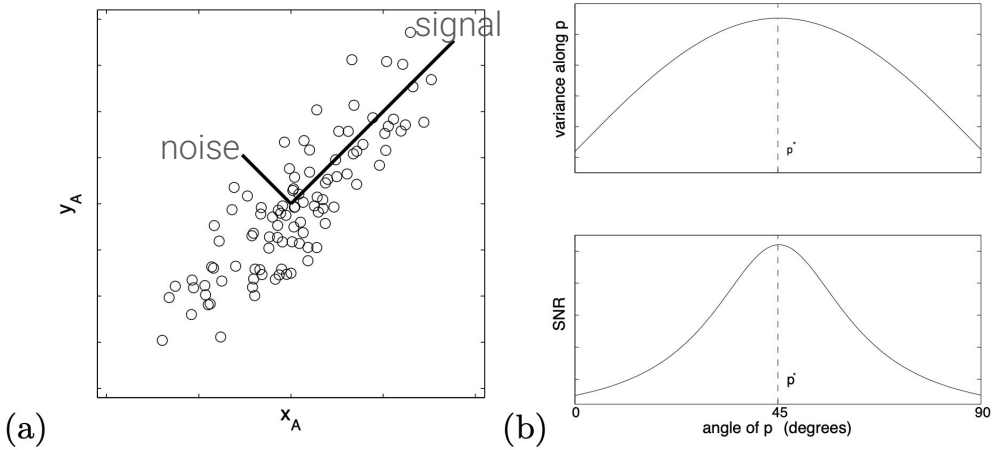


- Every deviation from the straight line must be noise
  - We can define $SNR = \frac{\sigma^2_{signal}}{\sigma^2_{noise}}$.

Rotation of input to improve S/N

Change of basis

- By rotating the axes we can find the optimal angle where **variance and SNR are maximized**
- At the same time we are **minimizing the redundancy** (reducing the correlation)



Formally: find the first principal component

- Let **X** be a data matrix with **n** samples and **p** features
  - We assume that each column is **centered**, i.e., that the mean of the column has been subtracted away from each entry
- To find the **first principal component $\varphi_1 = (\varphi_{11},\ldots,\varphi_{p1})$**, we solve the following optimization problem

$$\max_{\phi_{11},\ldots,\phi_{p1}} \left\{ \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{p} \phi_{j1} x_{ij} \right)^2 \right\}$$

projection of x on ϕ

$$\text{subject to} \sum_{j=1}^{p} \phi_{j1}^2 = 1.$$

Scalar projection of x_i onto ϕ_1. Also known as **the score** z_{i1}.

Sample mean of the scores is 0:

$\frac{1}{n}\sum_{i=1}^{n} z_{i1} = \sum_{j=1}^{p} \phi_{j1} \frac{1}{n}\sum_{i=1}^{n} x_{ij} = 0$ Sample variance of the scores z_{i1}

(i.e., sample variance of projections of datapoints onto ϕ_1).

Find the second principal component

- To find the **second principal component** $\varphi_2 = (\varphi_{12},\ldots,\varphi_{p2})$, we solve the following optimization problem

$$\max_{\phi_{12},\ldots,\phi_{p2}} \left\{ \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{p} \phi_{j2} x_{ij} \right)^2 \right\}$$

$$\text{subject to } \sum_{j=1}^{p} \phi_{j2}^2 = 1 \text{ and } \sum_{j=1}^{p} \phi_{j1}\phi_{j2} = 0.$$

orthogonal axis

First and second principal components must be orthogonal.

Equivalent to saying that the scores $(z_{11}, \ldots, z_{n1})$ and $(z_{12}, \ldots, z_{n2})$ are uncorrelated.

Solving the optimization problem

- These optimization problems are standard in linear algebra
- They are solved by the singular value decomposition (SVD) of **X**:

$$\mathbf{X} = \mathbf{U}\boldsymbol{\Sigma}\boldsymbol{\Phi}^T$$

- where the *i*th column of Φ is the *i*th principal component φ_i and the ith column of **UΣ** is the *i*th vector of scores $(z_{1i},\ldots,z_{ni})$
- The eigendecomposition of $\mathbf{X}^T\mathbf{X}$:

$$\mathbf{X}^T\mathbf{X} = \boldsymbol{\Phi}\boldsymbol{\Sigma}^2\boldsymbol{\Phi}^T$$

How many principal components are enough?

- We can think of the top principal components as **directions in space in which the data vary the most**

- The ith score vector ($z_{1i},\ldots,z_{ni}$) can be interpreted as a new variable
  - The variance of this variable decreases as we take *i* from 1 to p
  - However, **the total variance of the score vectors is the same as the total variance** of the original variables:

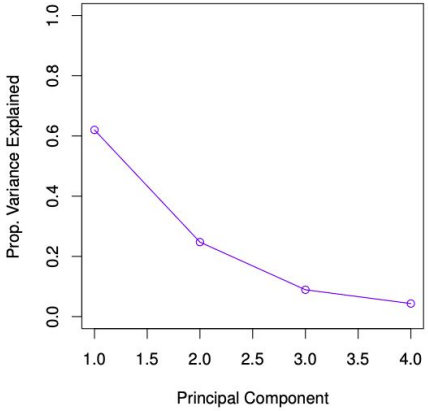
$$\sum_{i=1}^{p} \frac{1}{n} \sum_{j=1}^{n} z_{ji}^2 = \sum_{k=1}^{p} \mathrm{Var}(x_k).$$

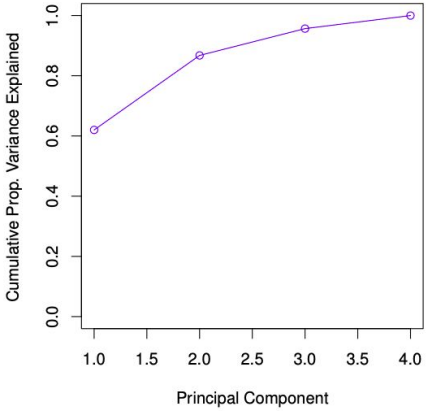
- We can quantify how much of the variance is captured by the first m principal components/score variables.

The proportion of variance explained

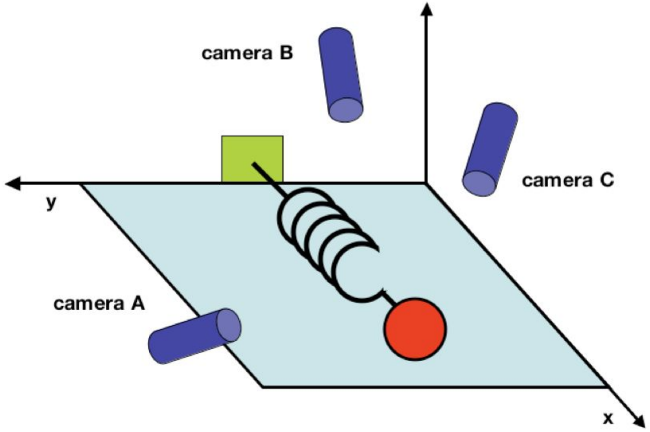
- The variance of the *m*th score variable is:

$$\frac{1}{n}\sum_{i=1}^{n} z_{im}^2 = \frac{1}{n}\sum_{i=1}^{n}\left(\sum_{j=1}^{p}\phi_{jm}x_{ij}\right)^2 = \frac{1}{n}\mathbf{\Sigma}_{mm}^2.$$





What would LASSO do instead?



Example on the IRIS dataset

- 4 available features from the IRIS dataset
  - sepal length, sepal width, petal length, petal width

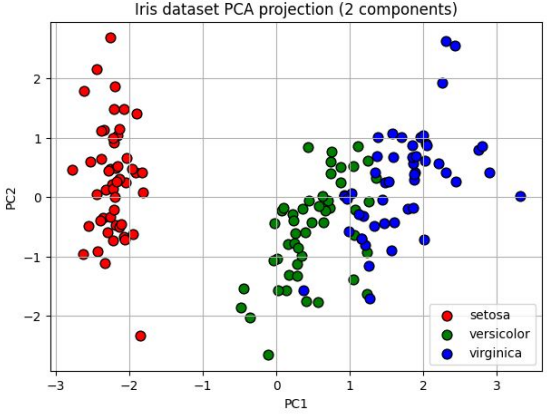
- Use PCA to **project this 4 feature** space into a **2-dimensional space**

Iris Versicolor **Iris Setosa** **Iris Virginica**

- Syntax:

```
from sklearn.decomposition  import PCA
X_reduced  =  PCA(n_components=2).fit_transform(iris.data)

scatter  =  ax.scatter(X_reduced[:, 0],  X_reduced[:, 1],
c=iris.target, s=40)
```



Beyond PCA: clustering

- **Goal**: Find groups of data points that are similar to one another
  - As in classification, we assign **a class or cluster label to each data point** in the data matrix
  - Unlike in classification, we do not have access to labeled training points: **we only observe the input variables**
  - Similarly to PCA, clustering is an **unsupervised** learning task

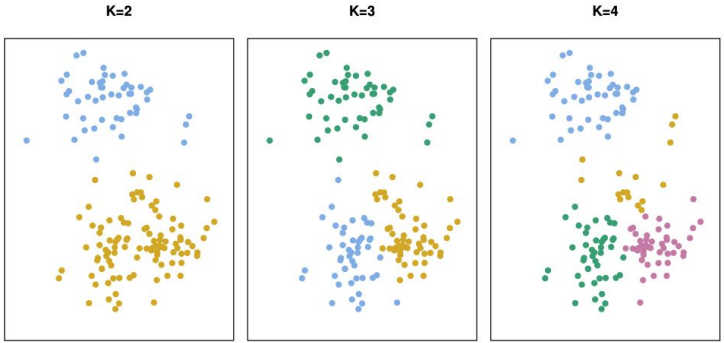
- We will discuss the **k-means clustering algorithm**

A form of PCA

K-means clustering

- **K** is the number of clusters and **must be fixed in advance**
- The goal of this method is to **minimize the dissimilarity of samples** $W(C_\ell)$ within each cluster C_ℓ:

$$\min_{C_1,\ldots,C_K} \sum_{\ell=1}^{K} W(C_\ell) \quad ; \quad W(C_\ell) = \frac{1}{|C_\ell|} \sum_{ij \in C_\ell} \text{Distance}^2(x_i, x_j).$$



Lloyd's algorithm for K-means clustering

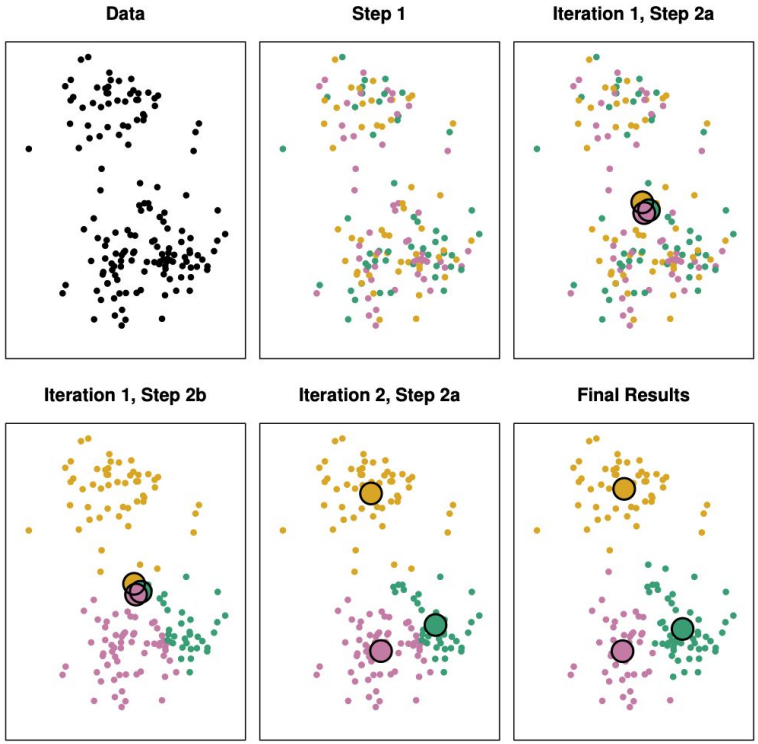
- **Assign each sample a cluster** label from 1 to K arbitrarily, e.g., uniformly at random

- Iterate these two steps until the clustering is constant:
  - **Find the centroid of each** cluster ℓ; i.e., the average x_ℓ of all the samples in the cluster:

$$\overline{x}_{\ell j} = \frac{1}{|C_\ell|} \sum_{i \in C_\ell} x_{ij} \quad \text{for } j = 1, \ldots, p.$$

  - **Reassign each sample to the nearest centroid**

In practice


Data
Step 1
Iteration 1, Step 2a
Iteration 1, Step 2b
Iteration 2, Step 2a
Final Results

Properties of K-means clustering

- Lloyd's algorithm always terminates (if we break ties consistently)

Why?

$$\min_{C_1,\ldots,C_K} \sum_{\ell=1}^{K} W(C_\ell)$$

$$W(C_\ell) = \frac{1}{|C_\ell|} \sum_{ij \in C_\ell} \text{Distance}^2(x_i, x_j) = 2 \sum_{i \in C_\ell} \text{Distance}^2(x_i, \overline{x}_\ell)$$

After each step of Lloyd's algorithm, K-means objective decreases or stays the same.

- Different initializations can yield different clusterings (solution returned **not guaranteed to be globally optimal**)

not "global minimum"

Questions when using clustering

- Is clustering appropriate? Could a **sample belong to more than one** cluster?
- **How many** clusters are appropriate?
  - Application dependent choices
- Are the clusters **robust**?
  - Run the clustering on different random subsets of the data. Is the structure preserved?
  - Try different clustering algorithms. Are the conclusions consistent?

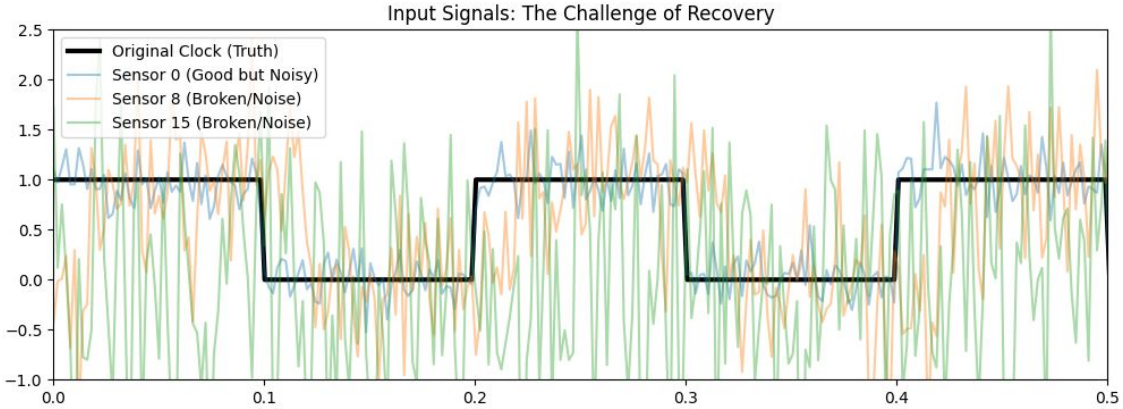
- Should we scale the variables before doing the clustering?
  - Variables with larger variance have a larger effect on the Euclidean distance between two samples
- Does Euclidean distance capture dissimilarity between samples?

Example…

Impact of regularization and dim reduction on a generated dataset

- Suppose we have 20 sensors monitoring a signal (squared wave)
  - 5 sensors are good, 5 are distorted, 10 are broken (pure noise)

- How are the features selected if the target is the clean clock signal?
  - See this notebook

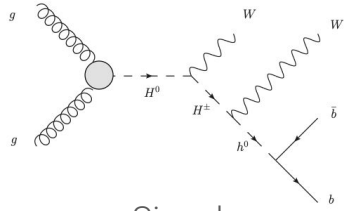


Hands on…

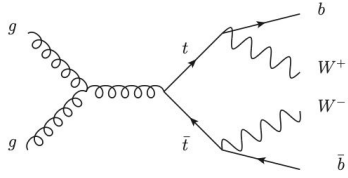
Separate higgs events from background events

- A dataset is available where **collision events are generated** using a MC simulation
- Signal and bkg events have the **same final state** → separation is possible based on the **kinematic features of the particles** in the final state
  - 2 W bosons and 2 b quarks
- **Goal:** consider the techniques seen today distinguish exotic Higgs events from bkg involving top quarks and measure the accuracy of the predictions
- **Questions:**
  - Do all the variables matter in the same way?
  - Since we are performing a classification task, does it make sense to use the Ridge/Lasso penalty with RSS or MSE?
- The dataset can be accessed directly with the following syntax:

```
from sklearn.datasets import fetch_openml
# Load the Higgs dataset
higgs_data = fetch_openml(name='higgs', version=1, as_frame=True)
df = higgs_data.frame
```



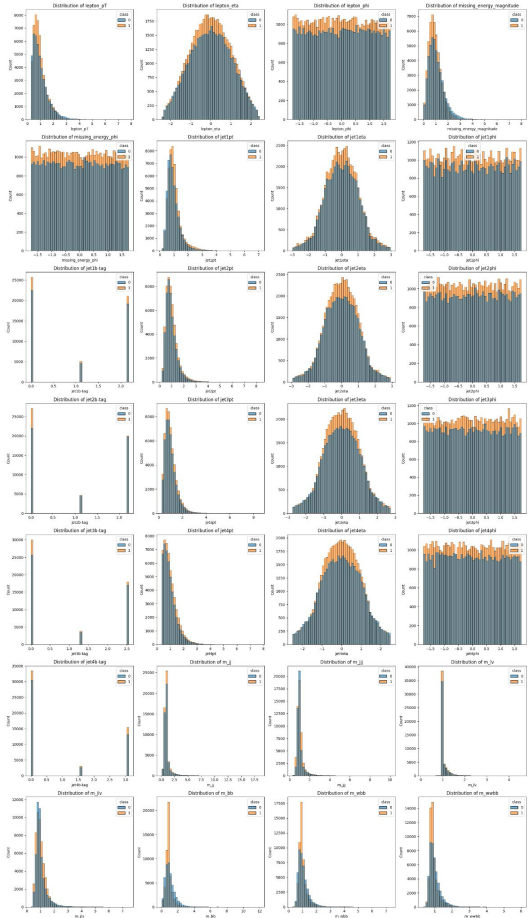
Signal



Background

Typical variables from an experiment at LHC

- More info on the dataset available
  - On the openml website
  - In the original paper from 2014

- Features include:
  - 21 low-level features (kinematic properties):
    - lepton pT, lepton eta, lepton phi, missing energy magnitude, missing energy phi, jet 1 pt, jet 1 eta, jet 1 phi, jet 1 b-tag, jet 2 pt, jet 2 eta, jet 2 phi, jet 2 b-tag, jet 3 pt, jet 3 eta, jet 3 phi, jet 3 b-tag, jet 4 pt, jet 4 eta, jet 4 phi, jet 4 b-tag
  - 7 high-level features derived by physicists:
    - m_jj, m_jjj, m_lv, m_jlv, m_bb, m_wbb, m_wwbb
  - The first column is the class label (1 for signal, 0 for background)

- To be noticed that the **features in the dataset have been scaled** wrt the original distributions
  - A lepton with a pt of 1GeV wouldn't even reach the calorimeters